



Technical Assessment

AI ENGINEER

Dual-Mode Agentic RAG Chatbot



1. Overview

This assessment evaluates a candidate's ability to design and ship a production-grade Generative AI system end to end. The brief is intentionally close to real EMB client work: building an LLM-powered chatbot that answers questions over a company's knowledge base using two distinct retrieval strategies within a single conversational agent.

The candidate is expected to make and defend their own architectural decisions. There is no single correct stack. We are assessing judgment, correctness, and engineering quality over polish.

2. The Assignment

Build a single chatbot that answers questions about a small fictional company using two different knowledge sources, and decides on its own which source to use for each question.

- **Unstructured knowledge (Agentic RAG):** answered via vector retrieval over a set of documents, with citations back to the source.
- **Structured knowledge (text-to-SQL):** answered by generating and executing a query against a structured data table. Note: structured data is not embedded for vector search; the agent writes a query against it.
- **Routing:** the agent must decide per question which tool to invoke, and may use both for questions that span documents and data.

3. Dataset Provided

A small, fixed dataset is supplied so submissions can be graded consistently against the same ground truth.

Source	Contents
Unstructured	3 to 5 documents (PDF/Markdown): HR leave policy, product FAQ, returns and refund policy.
Structured	One orders table (~200 rows): order_id, customer, product, amount, status, order_date.

All dataset files (documents and orders data) are available here: [Download dataset](#)

Treat the current date as 15 June 2026 for any time-based question.

4. Required Behaviour

- **Document questions** (e.g. "What is the refund window?") are answered via vector RAG with citations.
- **Data questions** (e.g. "What was total revenue last month?", "How many orders are pending?") are answered via text-to-SQL.
- **Mixed questions** (e.g. "Our policy allows 30-day returns; did order 1234 qualify?") use both sources.
- **Out-of-scope questions** must return a safe fallback ("I don't have that information"). No hallucinated policy text and no hallucinated SQL columns.

5. Technical Constraints

Layer	Requirement
Backend	FastAPI, with token-level response streaming.
LLM	Any provider (OpenAI, Claude, or Gemini). API key via environment variable.
Frontend	Next.js. Simple chat UI that shows which tool was used and the citation or generated SQL.
Vector store	Candidate's choice (pgvector, Pinecone, Supabase, Chroma, FAISS, etc.).
Deployment	Deployed anywhere publicly accessible. A live URL is mandatory.
Packaging	Dockerised. Public GitHub repository with README.

6. Deliverables

- Live, publicly accessible URL of the working chatbot.
- Public GitHub repository with all source code and a Dockerfile.
- README covering: architecture, choice of model / embeddings / vector store and the reasoning, how the routing decision works, and known limitations.